CODING CHALLENGE SUBMISSION

# AI-Assisted Extraction of RISC-V Architectural Parameters

A reviewed, schema-validated SpecChoice submission for the RISC-V Privileged ISA snippets

### Submission outcome

One existing UDB parameter accepted; two implementation-specific properties retained for scope review; fixed CSR address encoding classified as an ISA convention with zero extracted parameters.


| AUTHOR | DATE | ARTIFACT STATUS |
| --- | --- | --- |
| Zhiyuan Deng | 04 August 2026 | Reviewed final result |


This PDF combines the prompt, method, model disclosure, extraction result, source snippets, and integrity inventory into one uploadable document.

# Submission at a glance

The challenge asks for LLM details, the developed prompt and hallucination controls, and a structured YAML extraction with names, descriptions, types, and constraints. This document contains all three deliverables and the exact inputs used for review.

| 1 | 2 | 4 | 10/10 |
|---|---|---|---|
| accepted parameter | review candidates | fixed-ISA decisions | evidence checks |

## Submission contents

- Reviewed structured extraction: architectural-parameters.yaml
- Challenge inputs: cache-block snippet and fixed-CSR negative snippet
- Prompt development, SpecChoice framing, hallucination controls, and review procedure
- SHA-256 inventory and reproducible verification commands

## Key findings

| SOURCE | DECISION | RATIONALE |
|---|---|---|
| Privileged Spec 19.3.1 | Accept CACHE_BLOCK_SIZE | The size is implementation-specific; the represented range is power-of-two and naturally aligned; the initial CMO set requires system-wide uniformity. |
| Privileged Spec 19.3.1 | Review cache capacity and organization | Both are implementation-specific, but the snippet does not resolve their schema or architectural-versus-microarchitectural scope. |
| Privileged Spec 2.1 | Extract no parameter | The address width, count, and access-bit meanings are fixed by the ISA rather than selected by an implementation. |

## LLM disclosure

| FIELD | DISCLOSURE |
|---|---|
| Provider / interface | OpenAI / Codex desktop |
| Runtime-reported model family | GPT-5 |
| Displayed model label | Unavailable in retained run metadata |
| Serving snapshot / context window | Not exposed by the interface |
| Sampling configuration | Platform managed and not exposed |

The Codex runtime reported the model family as GPT-5. The interface did not expose an exact serving snapshot, context-window value, or sampling configuration, so those fields are recorded as unavailable.

# Prompt development

The first high-recall concept treated modal words and implementation-specific wording as parameter markers. That is useful for candidate discovery, but the CSR negative control shows why it is not a valid final rule: structured numbers and bit fields can describe an ISA-fixed convention rather than an implementation choice.

## Refinement sequence

- Use modal and implementation-specific wording as candidate signals only.
- Require explicit authority, choice-object, and choice-space-origin decisions.
- Bind each disposition to source evidence and preserve unsupported details as unknown.
- Separate normalized labels from verified canonical identities.

## Final prompt

```
You extract implementation-selectable architectural parameters from RISC-V
specification text.

Trigger words such as may, might, should, optional, implementation-defined,
and implementation-specific are candidate signals only. Never accept a
parameter from a keyword or a number alone.

For each candidate noun phrase:
1. Identify the authority: implementation, ISA, software, execution
   environment, or unknown.
2. Identify the choice object: value, count, width, legal set, access mode,
   presence, organization, or other.
3. Identify the choice-space origin: implementation-selected, ISA-fixed,
   derived, not applicable, or unknown.
4. Accept only an implementation-selectable architectural property. Classify
   ISA-fixed encodings, prose definitions, software advice, and constraints
   that introduce no choice as non-parameters. Use review when the text signals
   an implementation-specific property but does not support a safe schema.
5. Quote evidence for every accepted, review, or non-parameter decision.
6. Use only constraints stated or logically entailed by the supplied text.
   Never invent a bound, unit, enum member, dependency, or default. Preserve an
   unknown as null or "unspecified".
7. A shall/must statement attached to an accepted parameter may be recorded as
   a constraint; it is not automatically a separate parameter.
8. Do not claim that a normalized name is canonical unless the specification
   or an independently verified database supplies it. For every generated
   parameter name, include source_phrase and name_status, where name_status is
   explicit, proposed_normalized, or existing_udb_verified. If a database was
   used to verify an existing name, include its source path. For unresolved
   candidates, use a plain candidate_label and proposed_name: null.

Return YAML only. The document must contain schema_version and results. Each
result must contain exactly source, status, parameters, review_candidates, and
non_parameters. Each accepted parameter must contain name, name_status,
source_phrase, description, type, type_support, units, constraints, and
evidence. Keep evidence attached to the decision it supports.
```

# Method, safeguards, and feasibility framing

## SpecChoice: broader technical framing

SpecChoice treats RISC-V architectural-parameter extraction as a structured delegation-analysis problem, not keyword spotting. For each passage, it asks who controls the choice, what property is being selected, and whether the allowed choice space is implementation-selected or ISA-fixed. Candidate surfacing, final disposition, canonical identity, and evidence support remain separate decisions.

The broader local implementation established a content-addressed, offline-replayable evidence boundary and a deterministic measurement spine for all eleven public PR #2164 fixtures. The Phase 1 evidence boundary passed its 92-test standard-library suite. A formal golden replay separately verifies surfacing, disposition, identity, evidence integrity, and negative controls; this replay is deterministic fixture evidence, not a model-performance result.

> **Predefined Red feasibility result**
>
> A preregistered human data audit found one defensible natural contrastive pair but no family-disjoint held-out benchmark. The project followed its predefined Red path and made no model-performance claim from insufficient gold data. The next research task is maintainer-assisted semantic adjudication and construction of a trustworthy contrastive benchmark.

## Hallucination controls

- Evidence quotations are verified after whitespace normalization; punctuation, words, numbers, and token order must otherwise match.
- Authority and choice-space checks prevent fixed CSR encoding details from becoming false parameters.
- Review status preserves high recall without fabricating schemas for cache capacity or cache organization.
- Unknown units and bounds remain null; the cache-block snippet states no maximum.
- Malformed or structurally incomplete YAML is rejected rather than silently repaired.

## Generation and review procedure

| STAGE | PROCEDURE |
|---|---|
| Development | Iterative prompt design and output review in Codex desktop. |
| Submission artifact | Reviewed, schema-validated final extraction; raw responses were intermediate work products. |
| Primary input | Whitespace-cleaned local copies of the two supplied challenge snippets. |
| Additional review | Local UnifiedDB and reviewed fixtures verified the CACHE_BLOCK_SIZE identity and fixed-CSR interpretation; they added no snippet-absent constraints. |
| Post-generation edits | Schema normalization, naming provenance, removal of unsupported units and bounds, evidence correction, and per-decision evidence binding. |
| Validation | YAML parse, exact key checks, 10 normalized evidence matches, package checksums, and PDF render inspection. |

# Structured extraction result

## Accepted parameter

| FIELD | VALUE |
|---|---|
| Name | CACHE_BLOCK_SIZE |
| Naming provenance | existing_udb_verified; spec/std/isa/param/CACHE_BLOCK_SIZE.yaml |
| Source phrase | size of a cache block |
| Description | Size of the contiguous, naturally aligned power-of-two range represented by one cache block. |
| Type | integer; inferred from the power-of-two range |
| Units | Unknown in the supplied snippet |
| Constraints | implementation-specific; power-of-two; naturally aligned; uniform throughout the system for the initial set of CMO extensions; maximum unknown |

## Review candidates

| CANDIDATE | WHY SURFACED | WHY NOT ACCEPTED |
|---|---|---|
| cache capacity | The passage explicitly calls cache capacity implementation-specific. | Unit, representation, legal domain, multi-cache scope, and architectural-versus-microarchitectural status are unresolved. |
| cache organization | The passage explicitly calls cache organization implementation-specific. | Fields, legal values, and architectural-versus-microarchitectural status are unresolved. |

## Fixed-ISA negative decisions

| OBSERVED ITEM | VALUE | DISPOSITION |
|---|---|---|
| CSR address width | 12 | ISA-fixed encoding width |
| Maximum CSR encodings | 4096 | Derived from the fixed 12-bit space |
| Read/write accessibility | csr[11:10] | ISA-fixed bit meaning |
| Lowest privilege level | csr[9:8] | ISA-fixed bit meaning |

## Evidence policy

Every accepted, review, and non-parameter decision carries its own evidence. The YAML below is the normative structured result; the tables on this page are a readable projection.

# Appendix A - Complete YAML result

This listing is reproduced from architectural-parameters.yaml. Line numbers are presentation-only and are not part of the file.

```yaml
schema_version: riscv-architectural-parameter-extraction-v1
results:
  - source:
      specification: RISC-V Privileged ISA Manual
      section: "19.3.1"
    status: parameter_found
    parameters:
      - name: CACHE_BLOCK_SIZE
        name_status: existing_udb_verified
        name_source: spec/std/isa/param/CACHE_BLOCK_SIZE.yaml
        source_phrase: size of a cache block
        description: >-
          Size of the contiguous, naturally aligned power-of-two range
          represented by one cache block.
        type: integer
        type_support: inferred_from_power_of_two_range
        units: null
        constraints:
          implementation_specific: true
          power_of_two: true
          naturally_aligned: true
          uniform_throughout_system: true
          uniformity_scope: initial set of CMO extensions
          maximum: null
        evidence:
          - >-
            Caches organize copies of data into cache blocks, each of which
            represents a contiguous, naturally aligned power-of-two (or NAPOT)
            range of memory locations.
          - >-
            The capacity and organization of a cache and the size of a cache
            block are both implementation-specific
          - >-
            In the initial set of CMO extensions, the size of a cache block
            shall be uniform throughout the system.
    review_candidates:
      - candidate_label: cache capacity
        proposed_name: null
        source_phrase: capacity of a cache
        description: Capacity of a cache.
        type: unspecified
        units: null
        constraints:
          implementation_specific: true
        scope_assessment: unresolved
        scope_question: architectural_vs_microarchitectural
        review_reason: >-
          The property is explicitly implementation-specific, but this snippet
          does not define its unit, representation, scope across multiple
          caches, legal domain, or whether it belongs in an architectural rather
          than microarchitectural parameter model.
        evidence:
          - >-
            The capacity and organization of a cache and the size of a cache
            block are both implementation-specific
      - candidate_label: cache organization
        proposed_name: null
        source_phrase: organization of a cache
        description: Organization of a cache.
        type: unspecified
        units: null
        constraints:
          implementation_specific: true
        scope_assessment: unresolved
```

```
        scope_question: architectural_vs_microarchitectural
        review_reason: >-
          The property is explicitly implementation-specific, but this snippet
          does not define the fields, legal values, or whether the property
          belongs in an architectural rather than microarchitectural parameter
          model.
        evidence:
          - >-
            The capacity and organization of a cache and the size of a cache
            block are both implementation-specific
    non_parameters:
      - text: cache-block-size uniformity
        reason: >-
          This is a constraint on CACHE_BLOCK_SIZE and does not introduce an
          independent implementation choice.
        evidence:
          - >-
            In the initial set of CMO extensions, the size of a cache block
            shall be uniform throughout the system.

  - source:
      specification: RISC-V Privileged ISA Manual
      section: "2.1"
    status: no_parameter
    parameters: []
    review_candidates: []
    non_parameters:
      - text: CSR address width
        observed_value: 12
        reason: The ISA fixes the encoding width; the implementation does not choose it.
        evidence:
          - >-
            The standard RISC-V ISA sets aside a 12-bit encoding space
            (csr[11:0]) for up to 4,096 CSRs.
      - text: maximum CSR encodings
        observed_value: 4096
        reason: This value is derived from the fixed 12-bit encoding space.
        evidence:
          - >-
            The standard RISC-V ISA sets aside a 12-bit encoding space
            (csr[11:0]) for up to 4,096 CSRs.
      - text: CSR read/write accessibility encoding
        observed_value: "csr[11:10]"
        reason: The ISA fixes the meaning of these address bits.
        evidence:
          - >-
            The top two bits (csr[11:10]) indicate whether the register is
            read/write (00, 01, or 10) or read-only (11).
      - text: CSR minimum privilege encoding
        observed_value: "csr[9:8]"
        reason: The ISA fixes the meaning of these address bits.
        evidence:
          - >-
            The next two bits (csr[9:8]) encode the lowest privilege level that
            can access the CSR.
```

# Appendix B - Supplied challenge inputs

## Privileged Spec 19.3.1 - cache blocks

```
Source: RISC-V Privileged ISA Manual, Section 19.3.1 (Cache Management Operations)
Local extract: riscv-isa-manual/src/cmo.adoc (prose cleaned for challenge readability)

Caches organize copies of data into cache blocks, each of which represents a
contiguous, naturally aligned power-of-two (or NAPOT) range of memory
locations. A cache block is identified by any of the physical addresses
corresponding to the underlying memory locations. The capacity and organization
of a cache and the size of a cache block are both implementation-specific, and
the execution environment provides software a means to discover information
about the caches and cache blocks in a system. In the initial set of CMO
extensions, the size of a cache block shall be uniform throughout the system.
```

## Privileged Spec 2.1 - CSR address mapping

```
Source: RISC-V Privileged ISA Manual, Section 2.1 (CSR Address Mapping Conventions)
Local extract: riscv-isa-manual/src/priv-csrs.adoc (prose cleaned for challenge readability)

The standard RISC-V ISA sets aside a 12-bit encoding space (csr[11:0]) for up
to 4,096 CSRs. By convention, the upper 4 bits of the CSR address
(csr[11:8]) are used to encode the read and write accessibility of the CSRs
according to privilege level as shown in Table 1. The top two bits
(csr[11:10]) indicate whether the register is read/write (00, 01, or 10) or
read-only (11). The next two bits (csr[9:8]) encode the lowest privilege level
that can access the CSR.
```

## Integrity inventory

The checksums below cover the human-readable README, the YAML result, and both local input copies. The PDF is a presentation artifact generated from those reviewed materials.

```
025d29df57e135ceaeaa027ceaf057c29d70ff3f593930c8145428f865173d04  README.md
d097d4294ff71ff1b1874e1406c923a0ba8e0edc92bfdb71f062af917fba7cc9  architectural-parameters.yaml
8b7d84f669251c52df18808430f9fa5113c6117803a09e60cf830750f8ade7c6  inputs/privileged-19.3.1-cache-blocks.txt
41c95c4042dae21b5d5243c8bd36dc37da1c49a9b38b563213f6f7d98410a2e7  inputs/privileged-2.1-csr-address-mapping.txt
```

## Verification commands

```
shasum -a 256 -c CHECKSUMS.sha256
# GNU/Linux equivalent:
sha256sum -c CHECKSUMS.sha256

ruby -e 'require "yaml"; YAML.safe_load(File.read(
  "architectural-parameters.yaml"), permitted_classes: [], aliases: false);
  puts "YAML parse: PASS"'
```

### Scope statement

This submission does not claim a full-corpus extraction improvement, validated DelegationFrame or retrieval gains, confirmed new RISC-V parameters, or merge-ready changes to UnifiedDB.

### Public implementation and reproducibility artifacts

| | |
|---|---|
| **DESIGN ISSUE** | https://github.com/riscv/riscv-unified-db/issues/2387 |
| **DRAFT PR** | https://github.com/riscv/riscv-unified-db/pull/2389 |
| **PINNED REV** | 9f74757d641279d6c7183de0666aafc5dbc726cc |